Reporte Técnico — Fase 2

Gestión de Catálogo de Productos de Supermercado

Red Multi-Sucursal · Grafo + Dijkstra · Simulación con Qt

Contenido

1. Resumen Ejecutivo 3
2. Arquitectura del Sistema 3
2.1 Visión general 3
2.2 Modelo de datos multi-sucursal 3
2.3 Grafo de la red 3
3. Estructuras de Datos Implementadas 4
3.1 Estructuras de Fase 1 (por sucursal) 4
3.2 Estructuras nuevas de Fase 2 4
3.3 Algoritmo Dijkstra 4
4. Simulador de Transferencia 4
4.1 Diseño de la máquina de estados 4
4.2 Factor de aceleración 5
4.3 Señales emitidas 5
5. Interfaz Gráfica (Qt 6.11) 5
5.1 Estructura de la ventana 5
5.2 Sistema de estilos 6
6. Carga de Datos — CSV Multi-entidad 6
6.1 Orden obligatorio de carga 6
6.2 Validaciones por CSV 6
6.3 CSV de muestra incluidos 6
7. Decisiones de Diseño 6
7.1 Encadenamiento separado en la Hash 6
7.2 Catálogo independiente por sucursal 7
7.3 actualizarStock() en lugar de eliminar + reinsertar 7
7.4 señal redActualizada() como bus de comunicación 7
8. Resultados y Métricas 7

1. Resumen

La Fase 2 del proyecto extiende el sistema de gestión de catálogo de la Fase 1 transformándolo en una red multi-sucursal con interfaz gráfica Qt. Se implementaron 7 nuevas clases C++ y se integró la interfaz en una ventana de 7 pestañas que permite gestionar sucursales, visualizar el grafo de la red, realizar búsquedas con 4 estructuras distintas y simular transferencias de productos con tiempos reales escalados.

El proyecto cubre 9 estructuras de datos activas: Lista Simple, Lista Ordenada, Árbol AVL, Árbol B, Árbol B+, Tabla Hash (6 por catálogo) más Cola, Pila y Grafo con lista de adyacencia.

2. Arquitectura del Sistema

2.1 Visión general

La arquitectura sigue un modelo de capas. El núcleo (core/) conserva intactas las 6 estructuras de Fase 1. La capa de red añade Sucursal como contenedor de un catálogo independiente, RedSucursales como grafo que conecta las sucursales, y SimuladorTransferencia como máquina de estados que anima el flujo de productos.

Capa	Clases	Responsabilidad
core/	Producto, ListaSimple, ListaOrdenada, ArbolAVL, ArbolB, ArbolBPlus, TablaHash, Catalogo	6 estructuras + orquestador
Red	Sucursal, RedSucursales	Catálogo propio, grafo, Dijkstra
Flujo	Cola<T>, Pila<T>, Operacion	Colas de proceso, historial
Carga	CargadorRed	Parseo de 3 CSV
Simulación	SimuladorTransferencia	Máquina de estados Qt
GUI	MainWindow	7 tabs, señales redActualizada()

2.2 Modelo de datos multi-sucursal

Cada sucursal posee su propio Catálogo independiente con las 6 estructuras sincronizadas. Un mismo producto físicamente existe en la sucursal que lo alberga; si se transfiere parcialmente, el stock se resta en origen y se suma en destino usando actualizarStock() que actualiza las 6 estructuras simultáneamente.

2.3 Grafo de la red

RedSucursales implementa un grafo dirigido ponderado con lista de adyacencia. Cada arista tiene dos pesos independientes (tiempo en segundos y costo en quetzales), lo que permite a Dijkstra optimizar por cualquiera de los dos criterios en la misma ejecución sin cambiar la estructura.

3. Estructuras de Datos Implementadas

3.1 Estructuras de Fase 1 (por sucursal)

Estructura	Clave	Complejidad inserción	Complejidad búsqueda
Lista Simple	—	O(1) al frente	O(n) secuencial
Lista Ordenada	Nombre	O(n) posición correcta	O(n) con corte anticipado
Árbol AVL	Nombre	O(log n)	O(log n)
Árbol B (t=3)	Fecha caducidad	O(log n)	O(log n+k) rango
Árbol B+ (t=3)	Categoría	O(log n)	O(log n+k) categoría
Tabla Hash (101 buckets)	Código de barra	O(1) amortizado	O(1) amortizado

3.2 Estructuras nuevas de Fase 2

Estructura	Implementación	Uso en el sistema	Complejidad
Cola<T>	Lista enlazada FIFO	3 colas por sucursal: ingreso, traspaso, salida	O(1) todas las operaciones
Pila<T>	Lista enlazada LIFO	Historial de operaciones para rollback	O(1) todas las operaciones
Grafo (lista adyacencia)	vector<NodoGrafo*>	Red de sucursales con pesos tiempo y costo	O(V+E) recorrido

3.3 Algoritmo Dijkstra

Se implementó con cola de prioridad (min-heap) para obtener O((V+E) log V). El criterio es switcheable en tiempo de ejecución: el mismo método rutaOptima() optimiza por tiempo o por costo según el parámetro Criterio. El ETA total suma los pesos de las aristas más los tiempos de procesamiento de cada sucursal intermedia.

4. Simulador de Transferencia

4.1 Diseño de la máquina de estados

SimuladorTransferencia hereda de QObject y usa QTimer::singleShot encadenado para producir la secuencia de etapas sin bloquear el hilo principal de la GUI. Por cada sucursal en la ruta (excepto el destino final) se ejecutan cuatro etapas:

- INGRESO — el producto entra a la colaIngreso de la sucursal (duración: tiempoIngreso)
- TRASPASO — pasa a la colaTraspaso (duración: tiempoTraspaso)
- DESPACHO — pasa a la colaSalida (duración: tiempoDespacho)
- EN_VIAJE — viaja por la arista al siguiente nodo (duración: peso de la arista)

El nodo destino solo procesa INGRESO y luego se considera ENTREGADO. Los cambios reales al inventario (aplicarTransferenciaFinal) se ejecutan únicamente al completarse toda la ruta.

4.2 Factor de aceleración

Los tiempos del CSV están en segundos reales (p.ej. 30s para ingreso). El simulador escala cada intervalo: ms = (segundos × 1000) / factor. El usuario puede elegir entre 1x (tiempo real), 10x, 30x (default), 60x y 120x.

Factor	30s ingreso	25s arista	Total ruta 3 nodos (est.)
1x (real)	30s	25s	~4 min
10x	3s	2.5s	~25s
30x (default)	1s	0.8s	~8s
60x	0.5s	0.4s	~4s
120x	0.25s	0.2s	~2s

4.3 Señales emitidas

El simulador comunica su estado a la GUI exclusivamente a través de señales Qt, evitando acoplamiento directo:

```
etapaIniciada(sucursalId, etapa, mensaje)  →  actualiza badge de color
y sucursal actual
productoEnTransito(origen, destino)        →  actualiza etiqueta de
tránsito
simulacionCompletada(exitosa)              →  desbloquea controles,
emite redActualizada()
log(mensaje)                               →  agrega línea con
timestamp al panel de log
```

5. Interfaz Gráfica (Qt 6.11)

5.1 Estructura de la ventana

MainWindow organiza la aplicación en 7 pestañas con QTabWidget. La RedSucursales es el único estado compartido; los tabs se comunican exclusivamente a través de la señal redActualizada() que dispara un refresco en cascada de todos los combos, listas y tablas.

Tab	Contenido principal	Widgets Qt destacados
Sistema	Tarjetas de estadísticas, botones de carga CSV, tabla resumen	QFrame tarjetas, QFileDialog, QLabel dinámico
Sucursales	CRUD de sucursales, inventario AVL in-order con búsqueda	QListWidget, QSplitter, QFormLayout, QSpinBox
Red	Grafo dibujado con nodos y aristas, CRUD de conexiones	QGraphicsScene, QGraphicsView, QGraphicsEllipseItem
Inventario	Búsqueda por 4 criterios (AVL/Hash/B+/B), agregar/eliminar	QComboBox criterio, QTableWidget con setForeground

Transferencia	Cálculo de ruta, simulación animada, log en vivo	QProgressBar, QTextEdit log, QScrollArea panel
Rendimiento	Benchmark 4 estructuras con 4 casos y análisis	QTableWidget resultados, std::chrono medición
Visualización	Genera .dot y convierte a PNG via Graphviz	QProcess dot, QPixmap renderizado inline

5.2 Sistema de estilos

Se aplica una QPalette global blanca/negra al MainWindow para neutralizar el tema oscuro de Windows. Adicionalmente, cada widget recibe estilos explícitos a través de funciones estáticas (estiloTabla, estiloBoton, estiloCampo, estiloGroupBox, estiloLista) que fijan color de texto, fondo y bordes. Cada QTableWidgetItem recibe setForeground(QColor(TEXTO_NEGRO)) explícito.

6. Carga de Datos — CSV Multi-entidad

6.1 Orden obligatorio de carga

CargadorRed impone el orden: sucursales → conexiones → productos. Las conexiones requieren que ambas sucursales existan; los productos requieren que su sucursalId esté registrada. Si se invierte el orden, el cargador rechaza los registros y los registra en data/errors_red.log.

6.2 Validaciones por CSV

CSV	Validaciones aplicadas
sucursales.csv	Campos obligatorios no vacíos, tiempos enteros >= 0, ID duplicado
conexiones.csv	Origen y destino existen en la red, tiempo y costo numéricos positivos
productos_fase2.csv	sucursalId existe, código de barra exactamente 10 dígitos, precio >= 0, stock >= 0, nombre y categoría únicos por sucursal

6.3 CSV de muestra incluidos

- sucursales.csv — 7 sucursales guatemaltecas (SUC01 – SUC07) con tiempos de proceso realistas
- conexiones.csv — 10 conexiones (algunas bidireccionales, una unidireccional SUC05→SUC06)
- productos_fase2.csv — 1050 productos, 150 por sucursal, generados con generar_csv_fase2.py

7. Decisiones de Diseño

7.1 Encadenamiento separado en la Hash

Se eligió chaining sobre open addressing porque permite factor de carga > 1 sin degradación brusca, la eliminación no genera huecos que afecten búsquedas futuras, y la visualización de colisiones (cadenas visibles) es pedagógicamente más clara.

7.2 Catálogo independiente por sucursal

Cada sucursal posee sus propias 6 estructuras en lugar de un catálogo global con índice de sucursal. Esto garantiza que las búsquedas AVL y Hash de una sucursal sean O(log n) y O(1) sobre el inventario de esa sucursal exclusivamente, sin interferencia de las demás.

7.3 actualizarStock() en lugar de eliminar + reinsertar

Cuando se transfieren unidades parciales, reinsertar el producto modificado en las 6 estructuras generaría 6 eliminaciones + 6 inserciones = 12 operaciones O(log n) o O(n). actualizarStock() recorre las estructuras actualizando el campo en el nodo existente: 6 búsquedas O(1)/O(log n) con asignación directa, mucho más eficiente y sin riesgo de fallo en la reinserción.

7.4 señal redActualizada() como bus de comunicación

En lugar de pasar referencias entre tabs o usar un controlador central, cada tab se suscribe a redActualizada() y reconstruye sus widgets dinámicos cuando recibe la señal. Esto desacopla completamente los tabs: agregar una sucursal en el Tab Sucursales dispara automáticamente el refresco de los combos en Inventario, Transferencia, Rendimiento y Visualización sin que el código del Tab Sucursales sepa que esos tabs existen.

8. Resultados y Métricas

Métrica	Valor
Líneas de código total (sin data/docs)	~4800 (core) + ~2100 (nuevas) + ~1850 (GUI) = ~8750
Clases implementadas	17 (Fase 1) + 7 nuevas (Fase 2) = 24 total
Estructuras de datos activas	9 (6 por catálogo + Cola + Pila + Grafo)
Tabs en la interfaz	7
CSV de muestra incluidos	3 (sucursales, conexiones, productos)
Productos de prueba	1050 (150 por sucursal × 7 sucursales)
Sucursales de prueba	7 (Guatemala, Zona 18, Villa Nueva, Mixco, Xela, Petén, Escuintla)
Conexiones de prueba	10 (bidireccionales y unidireccionales)